AI-Native Retrieval Engine — Setup Guide

For FYP partners running the project from the shared zip (`FYP_Retrieval_Engine.zip` , 11 Oct 2026, commit `8b966ec`) · Windows 10/11, PowerShell · Last updated 2026-10-16

The zip is a full snapshot. It already contains the Qdrant index, the Ollama model and the reranker and BM25 models, so you do **not** need to re-ingest data or pull models. You install Python, create a virtual environment, update the code, and start three local services.

0. Prerequisites

Tool	Version	Notes
Windows	10 or 11, 64-bit	
Python	3.11.x, 64-bit, python.org build	Must be the **signed** python.org build (not Microsoft Store or `uv`), or SQLite/LangGraph imports can fail under Windows Smart App Control (see Troubleshooting)
Git for Windows	recent	Needed for step 2 (update to the latest code)
PowerShell	5.1 or 7.x	All commands below are PowerShell
Disk space	≈ 16 GB free	Zip 5.4 GB + extracted 8.1 GB + Python packages + embedding model. Delete the zip after extracting to get ~5 GB back
RAM	8 GB min, 16 GB comfortable	Close other heavy apps during experiment runs
GPU	Not required	Everything runs on CPU; the LLM (qwen2.5:3b) is small on purpose

> Install Python 3.11 from `python.org` , or run `winget install --id Python.Python.3.11 -e --scope user` . Check with `py -3.11 --version` .

1. Unzip to a short path

If you downloaded the zip (Drive, WhatsApp, browser), unblock it **before** extracting. Otherwise Windows marks every extracted file as "from the internet" and may block `qdrant.exe` / `ollama.exe` . Then extract to a short path. Long nested folders hit the Windows path-length limit (model files have long names).

```
Unblock-File .\FYP_Retrieval_Engine.zip
Expand-Archive .\FYP_Retrieval_Engine.zip -DestinationPath C:\fyp     # slow (8 GB); 7-Zip
is faster
cd C:\fyp\Retrieval_Engine
```

Run every command below from this folder.

2. Update to the latest code and set your Git identity

The zip is from 11 Oct. Newer commits are on GitHub, including the demo's final answer step and an evaluation fix. The zip's Git settings also carry Kabir's name, so set your own before committing.

```
git config user.name  "Your Name"
git config user.email "your-github-email@example.com"
git status                                  # should report a clean working tree
git pull origin k/controlled-path
```

You need access to `github.com/Kabir-Raj-01/Retrieval_Engine`. After pulling, `git log --oneline -1` should show a commit newer than `8b966ec`.

3. Create the virtual environment and install dependencies

The zip does **not** include `.venv` or `.env`. Create them locally so paths match your machine.

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.lock -e ".[dev]"
Copy-Item .env.example .env
```

> **Install from `requirements.lock`**, not `pip install -e ".[dev]"` alone. The lock pins exact versions. Newer releases of `uuid-utils`, `scikit-learn` and `pandas` contain native modules that Windows Smart App Control blocks, and the lock avoids them.

`.env` needs no edits to run the demo. It points at local Qdrant and leaves `LLM_API_KEY` empty, because local Ollama needs no key.

4. Start Qdrant (index already built)

The vector index (`var\qdrant_storage`) is already in the zip. This only starts the server.

```
$env:QDRANT__SERVICE__HOST = "127.0.0.1"
$env:QDRANT__STORAGE__STORAGE_PATH = "var\qdrant_storage"
$env:QDRANT__STORAGE__SNAPSHOTS_PATH = "var\qdrant_snapshots"
Start-Process -FilePath .\tools\qdrant\qdrant.exe -WindowStyle Minimized
Invoke-RestMethod http://127.0.0.1:6333          # prints the Qdrant version (v1.19.1)
```

5. Start Ollama (model already pulled)

`var\ollama` already contains `qwen2.5:3b`, so there is nothing to download.

```
$env:OLLAMA_MODELS = "$PWD\var\ollama"
$env:OLLAMA_HOST = "127.0.0.1:11434"
$env:OLLAMA_KEEP_ALIVE = "2h"
Start-Process -FilePath .\tools\ollama\ollama.exe -ArgumentList serve -WindowStyle
Minimized
.\tools\ollama\ollama.exe list                    # should show qwen2.5:3b
```

If it is missing, run `.\tools\ollama\ollama.exe pull qwen2.5:3b` (needs internet, ~1.9 GB).

6. Verify the install

```
pytest -m "not integration and not llm and not e2e"   # ~1,000 tests, ~3 min, all must pass
python -m engine.llm.smoke --role planner              # must end "ok: role=planner ..."
```

> The dense embedding model (`BAAI/bge-base-en-v1.5`) is **not** in the zip, because it lives in each machine's own Hugging Face cache. The first API request or eval run downloads it automatically (~440 MB, internet needed once) into `%USERPROFILE%\.cache\huggingface` .

7. Run the project

Window 1 — API server (activate the venv in every new window)

```
.\.venv\Scripts\Activate.ps1
python -m uvicorn engine.api.app:app --host 127.0.0.1 --port 8000
```

Swagger UI: `http://127.0.0.1:8000/docs`

Window 2 — the FYP-I demo

```
.\.venv\Scripts\Activate.ps1
$env:PYTHONIOENCODING = "utf-8"
python -m examples.demo_fyp1
```

This runs the default question (dev task multihop:0082) controller-OFF, then controller-ON, then answers it with the local Ollama model and passage citations (expected: `No[2][1]`). The first run of a question waits ~15–30 s for the planner. The first answer on CPU takes ~1–2 min; repeats take ~1 s while Ollama keeps the model loaded. Options: `--question "..."` , `--max-retrieval-calls N` , `--no-answer` . Save the output: `python -m examples.demo_fyp1 | Out-File -Encoding utf8 var\demo_output.txt` .

Optional — an experiment run

```
python -m eval.run --config configs\experiments\dense_oneshot_dev.yaml
python -m eval.report --run <run_id>
```

Keep Qdrant running for the whole run. A run that loses Qdrant aborts and is not reportable.

8. Stopping everything

```
Stop-Process -Name qdrant, ollama
```

Stop the API with Ctrl+C in its window.

9. Troubleshooting

Symptom	Fix
`Activate.ps1 cannot be loaded`	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, reopen PowerShell
`connection refused` on :6333 or :11434	Qdrant/Ollama is not running. Redo step 4 or 5, and check `Get-Process qdrant, ollama`
Windows blocks `qdrant.exe` / "Windows protected your PC"	The zip was not unblocked before extracting (step 1). Run `Get-ChildItem -Recurse .\tools | Unblock-File`
Every call is slow	A URL uses `localhost` instead of `127.0.0.1`. Windows tries IPv6 first, which adds a delay to every call
`DLL load failed… Application Control policy has blocked this file`	Smart App Control is blocking an unsigned native module. Re-create the venv with python.org Python 3.11 and reinstall strictly from `requirements.lock`. Do **not** turn Smart App Control off: it cannot be turned back on without reinstalling Windows
Missing `VCRUNTIME140.dll` (qdrant.exe or torch)	Install the Microsoft Visual C++ Redistributable (x64)
"path too long" during install	Enable Windows long paths (`LongPathsEnabled = 1`), run `git config --global core.longpaths true`, keep the repo in a short path (step 1)
`database is locked` (SQLite)	Run only one API server at a time. Stop any other one
`py -3.11` → "No suitable Python runtime found"	Only a non-python.org 3.11 is registered. Install python.org 3.11 (Prerequisites)
`git pull` fails with an authentication error or "repository not found"	Ask Kabir to add your GitHub account as a collaborator, then sign in when Git prompts you

10. What's in the zip vs. what you create

Already in the zip	Created by you / on first run
Qdrant binary + index (`tools\qdrant`, `var\qdrant_storage`)	`.venv` and `.env` (step 3)

Ollama binary + qwen2.5:3b (`tools\ollama` , `var\ollama`)	Dense embedding model in `%USERPROFILE%\.cache\huggingface` (one-time download)
Reranker + BM25 models (`var\models`)	Latest code (step 2)
MultiHop-RAG raw + processed data (`data\raw` , `data\processed`)	New entries in `var\*.sqlite3` as you run tasks
Past experiment runs (`eval\runs` , 2.3 GB, reference only; not needed for the demo)	